

Teaching Language Models to Think

A Critical Review of DeepSeek-R1

Reinforcement learning, test-time compute, emergent behavior, and open models

Abstract

For years, the recipe for a better language model was simple to state and expensive to follow: pre-train on ever more text, fine-tune on human-written examples, then align with human feedback. DeepSeek-R1 (DeepSeek-AI, 2025) broke from that script. Its authors showed that a model can learn to reason through long, careful chains of thought using reinforcement learning (RL) alone, rewarded only for getting verifiable answers right and never shown how a human would solve the problem. This review walks through how R1 was built, the unexpected behaviors that appeared along the way, such as self-correction and spending more effort on harder questions, and what its open release has meant for researchers and developers. We also report a small toy experiment on GRPO's group-relative advantage, and look at what the approach still can't do.

1. Introduction

For the last several years, progress in large language models has mostly meant making them bigger. Scaling laws gave the field a reliable rule of thumb: more compute, more data, and more parameters meant better results. The usual pipeline followed from it. A model is pre-trained on a huge body of web text to predict the next token, as with GPT-4 (Achiam et al., 2023) and Llama (Touvron et al., 2023). It is then fine-tuned on curated instruction–response pairs, and finally aligned with reinforcement learning from human feedback, or RLHF (Ouyang et al., 2022).

This pipeline produces fluent models, but it has a weak spot. A standard model writes its answer in one pass, token by token, with little room to stop, doubt itself, and try again. On hard maths, logic, or programming problems, an early wrong turn tends to stay wrong. The model simply keeps going.

One response has been to spend more effort at inference time instead of only during training. Self-consistency decoding (Wang et al., 2022) showed that sampling several reasoning paths and comparing them improves accuracy. OpenAI's o-series (OpenAI, 2024) pushed the idea much further, with models that reason at length before answering. But how those models were trained was never disclosed. DeepSeek-R1 mattered because it explained the method in detail, and released the models, so others could build on it.

2. From R1-Zero to R1

The DeepSeek team asked a simple question: if you give a base model a reward for correct answers and little else, will it work out how to reason on its own? They tested that in two stages.

2.1 R1-Zero: reinforcement learning with no SFT

R1-Zero started from DeepSeek-V3-Base and went straight into large-scale RL, skipping supervised fine-tuning entirely. For the learning algorithm, the authors used Group Relative Policy Optimization, or GRPO (Shao et al., 2024). Standard Proximal Policy Optimization (Schulman et al., 2017) needs a separate critic model to estimate how good each response is, which is costly in memory. GRPO avoids the critic. It samples a group of answers to the same prompt and scores each one against the group's average.

The rewards were deliberately plain and rule-based:

- **Accuracy.** Did the response reach the right final answer in a maths problem, or pass the unit tests for a coding task?
- **Format.** Did the model keep its reasoning inside the designated tags (<think> ... </think>)?

What happened next was the interesting part. Nobody told the model how to reason, yet its responses grew steadily longer and it began checking its own intermediate steps. It was far from polished, though. R1-Zero's output was hard to read, and it sometimes switched languages halfway through a thought.

2.2 R1: a four-stage pipeline

A model that reasons well but is awkward to talk to isn't much use, so the final R1 recipe adds structure around the RL core:

- Stage 1. Cold start.** A small set of carefully written long reasoning examples is used to fine-tune the base model. This gives it a clear, readable style before RL begins.
- Stage 2. Reasoning RL.** Large-scale GRPO training follows, focused on tasks with checkable answers: maths, code, and logic.
- Stage 3. Rejection sampling and general SFT.** To make the model useful beyond maths and code, the authors sample several hundred thousand responses from the reasoning checkpoint, keep the good ones, and mix them with ordinary non-reasoning data covering writing, role-play, and factual questions.
- Stage 4. Final RL for alignment.** A last round of RL tunes the model for helpfulness and harmlessness.

3. Behaviors That Emerged

The most discussed result in the paper isn't a benchmark score. It's what the model started doing without being asked. None of these behaviors were programmed. They appeared because they helped earn the reward, which suggests that RL can lead a model to new strategies and not just sharpen ones it already had.

It learned to think for longer when it needed to. Easy questions got short answers. Hard, multi-step ones got reasoning chains that ran to thousands of tokens. The model was, in effect, deciding for itself how much effort a problem deserved.

It had “aha moments.” Partway through training, the model began to catch itself. Faced with a contradiction, it would write something like “Wait, let me re-read the problem,” then back up and try a different approach. It's tempting to call this human-like metacognition. Whether it reflects anything like real understanding is still an open question, and the paper doesn't claim to settle it.

4. Opening It Up: Distillation and Access

R1's influence goes beyond the technique itself. Training a frontier reasoning model from scratch takes enormous computing resources that few groups have. DeepSeek showed a cheaper route. Take the reasoning traces that R1 produces and use them to fine-tune much smaller models.

The team did this with Qwen and Llama models ranging from 1.5 billion to 70 billion parameters. The distilled versions performed well on competitive reasoning benchmarks, in some cases matching or beating much larger systems. Because the weights were released publicly, a small company, a university lab, or a single developer could run a capable reasoning model locally, without paying for a commercial API.

5. A Small Experiment: Does the Group-Relative Baseline Help?

The R1 paper's main argument for GRPO is practical: dropping the critic model saves memory. A natural follow-up question is whether the group-relative advantage also helps the learning itself. We tested this on a deliberately tiny problem, so that the comparison could be run many times and every part of it could be inspected.

5.1 Setup

There are eight “prompts”, each with one hidden correct answer: a sequence of three tokens drawn from a vocabulary of six, so 216 possible answers. The policy is a table of independent softmax probabilities, one per prompt and position. It is not a neural network. The reward is binary and rule-based, like R1's accuracy reward: 1 for an exact match and 0 otherwise. At each update we sample 16 answers per prompt and take one policy-gradient step. We compared three ways of computing the advantage:

- **REINFORCE (Williams, 1992).** The advantage is the raw reward, with no baseline.
- **REINFORCE with a value baseline.** A running per-prompt average of reward is subtracted, a simple stand-in for a learned critic.
- **GRPO.** Each answer's reward is compared with the mean of its own group and divided by the group's standard deviation.

The measure is the number of updates until the success rate, averaged over 10 updates, first reaches 90%, capped at 600. Each configuration used 30 random seeds, each with new hidden answers. Because the methods respond differently to step size, we swept five learning rates rather than tuning one per method. The whole experiment is a short NumPy script (provided separately).

5.2 Results

Table 1. Updates needed to reach 90% success (mean $\pm$ SD over 30 seeds). “>600” means no run reached 90% within 600 updates.

Learning rate	REINFORCE	REINFORCE + value baseline	GRPO
0.1	>600	>600	475 $\pm$ 18
0.3	504 $\pm$ 26	503 $\pm$ 26	182 $\pm$ 17
1	176 $\pm$ 20	175 $\pm$ 20	76 $\pm$ 14
3	81 $\pm$ 16	81 $\pm$ 16	51 $\pm$ 16
10	51 $\pm$ 16	51 $\pm$ 16	45 $\pm$ 16

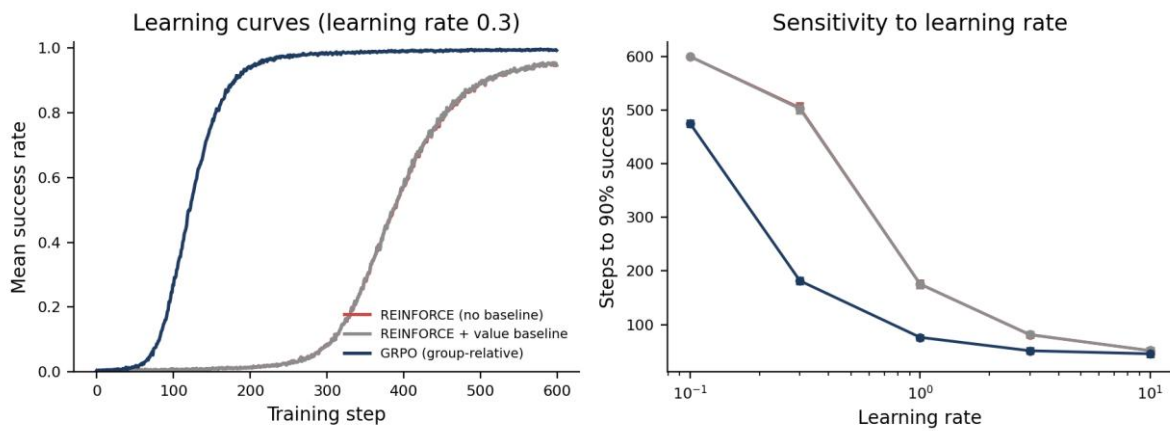


Figure 1. Left: mean success rate during training at learning rate 0.3. Right: updates to reach 90% success across learning rates (mean of 30 seeds, error bars are 95% confidence intervals; a value of 600 means the threshold was not reached).

GRPO reached 90% success faster than both alternatives at every learning rate. At a learning rate of 0.3 it took about 182 updates against roughly 504, and at 1 it took 76 against 176. At 0.1, neither REINFORCE variant got there within 600 updates in any run, while GRPO needed 475. The gap shrank as the learning rate grew, and at 10 the three methods were close (45 against 51). The value baseline was indistinguishable from no baseline. That is expected here, because successes are rare and the running average stays close to zero.

5.3 How to read this

The result is real but easy to over-interpret. Dividing by the group's standard deviation makes the advantage of a rare success large: with one correct answer in a group of 16, it is about four times the raw reward. In effect, that works like a larger learning rate. The numbers fit this reading. GRPO at a learning rate of 1 (76 updates) is close to plain REINFORCE at 3 (81 updates). So most of the speedup here may come from rescaling and not from a lower-variance gradient. We did not run the ablation that would separate the two, such as mean-centering without the standard-deviation division, so this remains a hypothesis.

The setting is also far from R1's. The policy is a lookup table, the task has one step and a binary reward, and there is no KL penalty, clipping, or long chain of thought. The experiment says

nothing about whether reasoning emerges. It illustrates one design choice in isolation, and a larger test would need real language models.

6. Limitations

R1 is impressive, but it isn't a solved problem. Three limitations stand out:

- **It needs answers that can be checked.** The whole approach leans on tasks where a program can say right or wrong, such as a numerical result or a passing test. Creative writing, political argument, and ethics have no such referee, and nobody yet has a reward signal as reliable for them.
- **Language mixing.** In long reasoning chains the model can drift from one language to another, starting in English and slipping into Mandarin or Spanish. This looks like an artifact of the training data, and it still needs better safeguards.
- **Cost and delay.** Thinking at length means generating thousands of tokens before the answer appears. That raises both the wait and the bill, so depth of reasoning always trades off against speed.

7. Conclusion

DeepSeek-R1 showed that strong multi-step reasoning doesn't have to be taught through mountains of human-written examples. It can be encouraged with a clear reward and enough training. That changes how we think about building capable models, and the open release put the method in the hands of far more people than a closed one would have.

The next steps seem fairly clear. Researchers will want reward signals that work outside maths and code, ways to get the same reasoning from fewer tokens, and reasoning models that can operate inside larger autonomous systems. As the open community builds on this work, the distance between proprietary labs and everyone else keeps shrinking.

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. (2023). GPT-4 Technical Report. arXiv preprint arXiv:2303.08774.
- DeepSeek-AI (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv preprint arXiv:2501.12948.
- OpenAI (2024). OpenAI o1 System Card. arXiv preprint arXiv:2412.16720.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems (NeurIPS)*, 35, 27730–27744.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., & Guo, D. (2024). DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv preprint arXiv:2402.03300.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M. A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. (2023). LLaMA: Open and Efficient Foundation Language Models. arXiv preprint arXiv:2302.13971.
- Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., & Zhou, D. (2022). Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv preprint arXiv:2203.11171.
- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4), 229–256.